

NetSage AI

AI-Assisted Cisco Network Troubleshooting Helper with Human Review

Project Domain: Cisco Lab / Packet Tracer	Architecture: Hybrid (Rules + LLM + HITL)
Safety Rule: Mandatory Human Review	Deliverables: Dataset (32), Prompts, Python Checker, UI, RAI Log

1. Executive Overview & Problem Statement

In computer networking labs and enterprise infrastructure, junior engineers frequently struggle to correlate symptoms with true root causes. For example, when a workstation receives an IP address but fails to reach an internal server, the underlying failure could originate from an 802.1Q subinterface tag mismatch, a missing default gateway, an inverted ACL wildcard mask, a DHCP pool exhaustion, or a missing NAT overload statement.

While Generative AI can assist in troubleshooting, **autonomous AI deployed in mission-critical networking poses severe operational risks**: Large Language Models (LLMs) frequently hallucinate non-existent CLI syntax, confuse standard subnet masks with Cisco inverted wildcard masks, diagnose Layer 3 routing faults before checking Layer 1 link state, and recommend disruptive actions such as device reloads during business hours.

NetSage AI resolves this dilemma through a **Hybrid Tri-Tier Architecture**: combining fast deterministic Python regex rule validation, evidence-backed AI diagnostic reasoning, and a mandatory **Human-in-the-Loop (HITL) review workflow**.

2. System Architecture & The 3-Tier Pipeline

Tier / Pipeline Stage	Mechanism & Functionality	Safety & Operational Purpose
Tier 1: Deterministic Rule Checker	Python AST/Regex parsing of raw CLI show-commands (<code>`show ip int brief`</code> , <code>`show ip route`</code> , <code>`show access-lists`</code> , <code>`show vlan brief`</code> , etc.).	Instant, 100% deterministic detection of syntax errors, admin down interfaces, inverted wildcards, and missing gateways.
Tier 2: Evidence-Backed AI Engine	Structured JSON prompting enforcing OSI Layer classification (L1–L7), confidence scoring (0–100%), verbatim CLI quotes, and Cisco fix scripts.	Correlates multi-protocol context and symptoms while eliminating hallucinations by requiring textual CLI line evidence.
Tier 3: Human-in-the-Loop Review	Interactive reviewer console: Accept , Edit (live CLI script editor), or Reject with automated Responsible AI audit logging.	Guarantees zero unverified configuration changes reach production devices; captures AI failure modes for continual safety improvement.

3. Clean Folder Structure (`backend/` & `frontend/`)

The project has been modularized cleanly into decoupled **backend** and **frontend** architectures:

Folder / Component	Files Contained	Description & Responsibilities
<code>`backend/`</code>	<code>`app.py`</code> <code>`engine/rule_checker.py`</code> <code>`engine/ai_engine.py`</code> <code>`data/cases.json`</code> <code>`data/responsible_ai_log.json`</code> <code>`prompts/diagnose_prompt.md`</code> <code>`tests/test_api.py`</code>	Flask REST API server, deterministic regex rule engine, hybrid AI diagnostic synthesizer, dataset persistence, prompt templates, and automated unit test suite.
<code>`frontend/`</code>	<code>`templates/index.html`</code> <code>`static/css/style.css`</code> <code>`static/js/app.js`</code> <code>`static/js/charts.js`</code>	Single-Page Application (SPA) with obsidian cyber glassmorphism design, interactive troubleshooting workbench, Chart.js analytics graphs, fix simulator, and review hub.
Root Deliverables	<code>`cases.csv`</code> <code>`rule_checker.py`</code> <code>`diagnose_prompt.md`</code> <code>`responsible_ai_log.md`</code> <code>`explanation.pdf`</code> <code>`README.md`</code>	Project root entrypoints and official submission deliverables for grading and verification.

4. Comprehensive Lab Dataset (32 Scenarios in `cases.csv`)

NetSage AI includes **32 authentic Packet Tracer lab scenarios** (exceeding the 30-case specification). Each scenario includes multi-line Cisco CLI show commands, topology notes, observed symptoms, expected root causes, OSI layers, severity ratings, and exact Cisco IOS remediation commands.

Concept Category	Cases Count	Representative Fault Scenarios Covered
VLAN & Trunking	6 Cases	Router-on-a-stick dot1Q tag mismatch, Native VLAN mismatch, missing switch VLAN in DB, trunk allowed VLAN pruning, static access on trunk.
Default Gateway	3 Cases	Missing `ip default-gateway` on L2 switch, host gateway outside subnet, gateway IP typo on host.
DHCP Services	4 Cases	DHCP pool 100% exhaustion (/28 mask), missing `ip helper-address` on router subinterface, DHCP snooping untrusted uplink drop.
DNS Resolution	3 Cases	`no ip domain-lookup` globally disabled, wrong DNS server IP configured in DHCP pool, DNS server unreachable.
Routing Protocols	7 Cases	Missing default static route (0.0.0.0/0), unreachable next-hop IP, OSPF Area ID mismatch, OSPF passive-interface hello drop, OSPF timer mismatch, RIP v1 vs v2, EIGRP AS mismatch.
Access Control Lists	3 Cases	Inverted wildcard mask (`255.255.255.0` in ACL), implicit deny dropping DNS return UDP traffic, standard ACL placed inbound on source interface.
NAT / PAT	3 Cases	Missing `overload` keyword on NAT pool (single IP exhaustion), missing `ip nat outside` on WAN interface, NAT pool range overlapping WAN link.
STP & Wireless	3 Cases	Port-security err-disable MAC violation, BPDU guard triggered port shutdown, duplex mismatch late collisions, WLAN SSID VLAN tag leak.

5. Step-by-Step Diagnostic & Human Review Workflow

- **Step 1: Ingestion & CLI Parsing:** The network engineer selects a lab case or pastes raw `show` command outputs (e.g. `show ip interface brief`, `show ip route`, `show access-lists`) and symptom notes into the Workbench.
- **Step 2: Deterministic Pre-Check:** The Python Deterministic Rule Engine scans the text for known configuration errors (such as `administratively down`, `err-disabled`, inverted ACL wildcards, or missing default gateways), returning instant findings with exact CLI line citations.
- **Step 3: Structured AI Reasoning:** The AI Diagnostic Engine applies the structured JSON prompt schema to classify the OSI layer, quote supporting CLI evidence, compute confidence, and produce the Cisco IOS configuration script.
- **Step 4: Packet Tracer Sandbox Simulation:** The user tests the remediation script in the virtual execution sandbox, which applies the commands in sequence and verifies reachability with simulated ICMP echo requests (100% pass).

- **Step 5: Mandatory Human Review (HITL):** A certified engineer reviews the diagnosis. The engineer selects ****Accept****, ****Edit**** (modifying commands to prevent side-effects), or ****Reject****. The submission is automatically written to ``responsible_ai_log.json`` and updates the live analytics dashboard.

6. Responsible AI: 5 Documented AI Failure Incidents

A cornerstone requirement of NetSage AI is documenting failure modes where AI was corrected by human network engineers:

Incident & Case	AI Diagnosis / Failure Mode	Human Engineer Correction & Implemented Safeguard
RAI-001 (CASE-006: ACL Mask)	Subtle Mask Hallucination: AI claimed rule 10 was missing <code>`established`</code> keyword, ignoring that <code>`255.255.255.0`</code> was entered instead of wildcard <code>`0.0.0.255`</code> .	Correction: Engineer replaced subnet mask with wildcard <code>`0.0.0.255`</code> . Guardrail: Added <code>`RULE-ACL-001`</code> deterministic check flagging <code>`255.255.255.x`</code> in ACLs.
RAI-002 (CASE-012: Admin Down)	Layer Skip: AI diagnosed a complex Layer 3 OSPF routing failure when interface GigabitEthernet0/24 was simply <code>`administratively down`</code> .	Correction: Engineer executed <code>`no shutdown`</code> on interface. Guardrail: Enforced Layer 1 interface status pre-checks before running L3 diagnostics.
RAI-003 (CASE-007: NAT PAT)	Incomplete Command Sequence: AI suggested adding <code>`overload`</code> keyword without negating old rule or clearing active NAT translation sessions.	Correction: Engineer added <code>`clear ip nat translation *`</code> to avoid CLI config lock. Guardrail: Augmented NAT remediation templates with session purge steps.
RAI-004 (CASE-003: DHCP Pool)	Disruptive Outage Risk: AI recommended reloading the router (<code>`reload`</code>) during production hours for a DHCP pool exhaustion issue.	Correction: Engineer expanded subnet to /24 and cleared active bindings with <code>`clear ip dhcp binding *`</code> . Guardrail: Added risk blocker intercepting <code>`reload`</code> .
RAI-005 (CASE-001: Subif VLAN)	Cisco CLI Side-Effect Omission: AI changed encapsulation tag to 20 without re-applying the subinterface IP address.	Correction: Engineer re-applied <code>`ip address 192.168.20.1 255.255.255.0`</code> (which Cisco IOS automatically deletes when dot1Q tag changes). Guardrail: Coupled subif encapsulation fixes with IP re-assignment.

7. Quick Start & Execution Commands

1. **Launch Web Dashboard:** ``python app.py`` → Navigate to ``http://127.0.0.1:5000``
2. **Run Deterministic CLI Checker:** ``python rule_checker.py`` or ``python rule_checker.py CASE-006``
3. **Execute Unit Test Suite:** ``python -m unittest discover backend/tests`` (15/15 passing)

Conclusion: *NetSage AI proves that combining deterministic Python checks, evidence-quoted LLM reasoning, and mandatory human review delivers an exceptionally robust, safe, and educational troubleshooting platform for Cisco Packet Tracer labs.*